

Software Engineering II

Event-Driven-Architecture

Mathis Neunzig
ABAP Developer at SAP
dhbw@mathis-neunzig.de



Architekturmuster

Was wir heute in der VL machen
ist grün markiert!

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

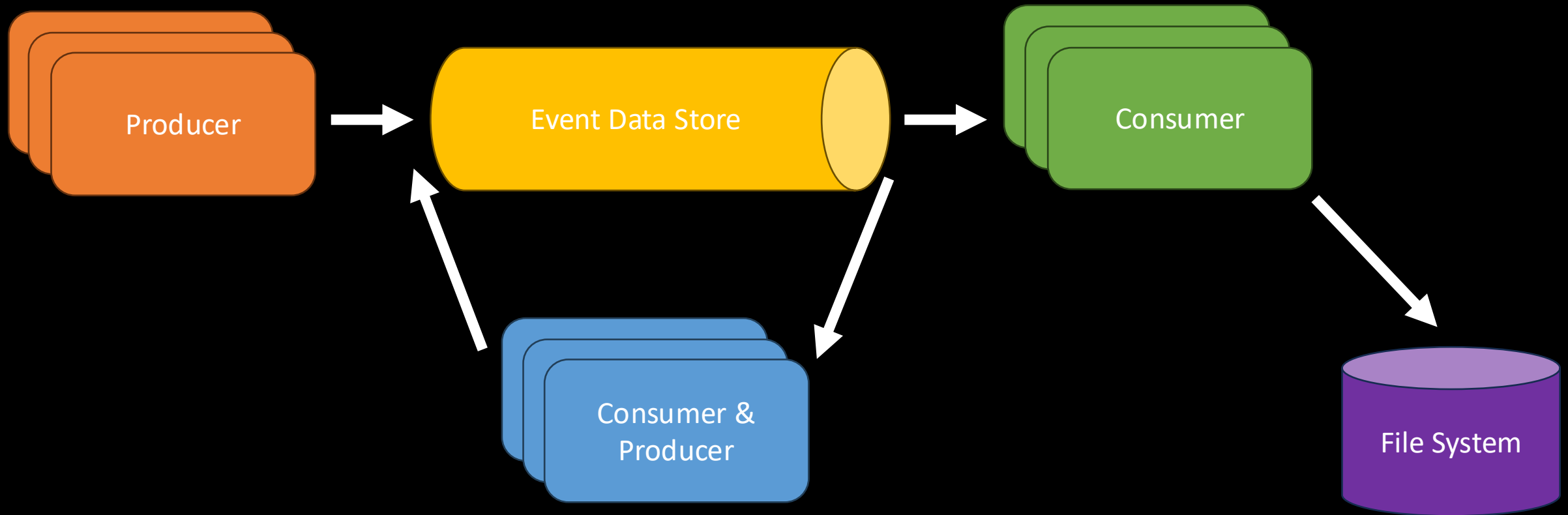
Event-Driven-Architecture

- Architektur, die auf Events als zentrale Kommunikationsform setzt
- Events sind Statusänderungen oder wichtige Vorfälle im System
- Ziel: lose Kopplung, bessere Skalierung, höhere Flexibilität
- Moderne Anwendungen nutzen EDA in Microservices, Cloud-Lösungen
- Beispiele: Onlineshop-Bestellungen, IoT-Sensoren, Finanztransaktionen
- Asynchrone Verarbeitung entlastet Nutzer-Interaktionen

Event

- Ereignis mit fachlich relevanten Daten, z.B. „Bestellung abgeschlossen“
- Besteht aus Event-Typ und Nutzlast (z.B. ID, Zeit, Benutzer)
- Wird von einem Producer erzeugt und versendet
- Wird von einem Consumer verarbeitet
- Beispiel: Benutzer hat sich eingeloggt, neue Nachricht eingegangen
- Kann als JSON, XML oder Binärformat vorliegen

Event



Observer Pattern

- Producer benachrichtigt mehrere Consumer bei Statusänderungen
- Consumer reagieren unabhängig voneinander auf Events
- Producer muss Consumer nicht kennen (lose Kopplung)
- GUI-Frameworks (z.B. Swing, WPF) nutzen Observer für UI-Events
- Grundlage für Event-Broker in verteilten Systemen
- Erleichtert Erweiterung durch neue Event-Reaktionen

Vorteile

- Lose Kopplung vereinfacht Erweiterungen und Änderungen
- Asynchrone Verarbeitung entlastet den Producer
- Skalierbarkeit durch unabhängige Consumer möglich
- Flexibel: neue Events oder Consumer leicht hinzufügbare
- Robustheit durch isolierte Services (z.B. Lager-Service, E-Mail-Service)
- Beispiele: neue Microservices in einem Webshop ohne Umbauten

Nachteile

- Eventual Consistency: Daten sind nicht sofort synchron
- Fehlersuche (Debugging) aufwendiger durch asynchrone Abläufe
- Event-Dokumentation entscheidend für Zusammenarbeit
- Testen schwieriger: Consumer müssen getrennt geprüft werden
- Monitoring & Logging nötig, um Events nachzuvollziehen
- Beispiel: verloren gegangene Events bei Netzwerkproblemen

Bestandteile

- **Event:** beschreibt was passiert ist (z.B. „Zahlung eingegangen“)
- **Producer (Producer):** sendet Event an Broker
- **Consumer (Consumer):** reagiert auf Event und verarbeitet es
- **Message Broker:** vermittelt Events zwischen Producern und Consumern
- **Event Store (optional):** speichert Events dauerhaft
- **Payload:** enthält Details wie IDs, Beträge, Zeitstempel

Beispiel: Webshop

- Event: „Bestellung abgeschlossen“ mit ID, Benutzer-ID, Preis
- Producer: Bestellsystem veröffentlicht Event an Event Store
- Consumer 1: Lager-Service prüft Bestände und reserviert Artikel
- Consumer 2: Rechnungsservice erstellt Rechnung
- Consumer 3: E-Mail-Service sendet Bestätigung an Kunden
- Jeder Service kann unabhängig skalieren und arbeiten

Technologien für EDA

- Apache Kafka: Event-Streaming-Plattform für große Datenmengen
- RabbitMQ: klassischer Message Broker mit Routing-Features
- AWS EventBridge, Azure Event Grid: Cloud-native Broker-Lösungen
- SAP Event Mesh: speziell für SAP-Landschaften
- MQTT für IoT-Anwendungen (z.B. Sensoren, smarte Geräte)
- Beispiele: Webshop-Events über Kafka, IoT-Daten über MQTT

Apache Kafka

- Kafka ist eine verteilte Event-Streaming-Plattform
- Entwickelt 2011 von LinkedIn
- Ursprünglich für Log-Analyse und Datenpipelines
- Heute auch für Microservices
- Speichert Events in Topics, die partitioniert und repliziert werden können
- Hochperformant, für Millionen Events pro Sekunde geeignet
- Unterstützt Streaming-Analysen mit Kafka Streams und ksqlDB
- Beispiele: E-Commerce-Ereignisse, Log-Analysen, IoT-Datenströme

Apache Kafka

- Producer schreibt Events in Topics, z. B. „orders“ oder „payments“
- Consumer lesen Events aus Topics in eigenem Tempo
- Topics sind in Partitionen aufgeteilt, für parallele Verarbeitung
- Kafka speichert Events dauerhaft, nicht nur zur sofortigen Verarbeitung
- Replikation sichert Daten gegen Ausfälle einzelner Broker
- Kafka eignet sich für Event Sourcing, große Datenmengen und verteilte Systeme

Apache Kafka

- Producer: Bestellsystem schreibt Event „order.completed“ in Topic „orders“
- Consumer 1: Lager-Service prüft Bestände und reserviert Ware
- Consumer 2: Rechnungs-Service erstellt Rechnung und sendet „invoice.created“ zurück
- Consumer 3: Versand-Service startet Versandvorbereitung und meldet „shipping.started“
- Kafka speichert alle Events, damit Consumer auch später noch Events verarbeiten können
- Beispiel-Event: {"orderId": 987, "userId": 456, "total": 49.99, "timestamp": "2025-05-27T12:15:00Z"}

RabbitMQ

- RabbitMQ ist ein Message Broker, der auf AMQP (Advanced Message Queuing Protocol) basiert
- Entwickelt 2007 von LShift und Rabbit Technologies, später von VMware übernommen
- Arbeitet mit Exchanges, Queues und Bindings für flexibles Routing
- Producer sendet Messages an Exchanges, nicht direkt an Queues
- Messages können persistent gespeichert werden, um Ausfallsicherheit zu gewährleisten
- Geeignet für Anwendungen mit vielen gleichzeitigen, kleinen Events, z. B. Benachrichtigungen

RabbitMQ

- Producer veröffentlicht Events an einen Exchange, z. B. „UserEvents“
- Exchange entscheidet, in welche Queue(s) das Event weitergeleitet wird
- Consumer abonniert Queues und liest Messages ab, wann immer er bereit ist
- Exchanges können Nachrichten an mehrere Queues gleichzeitig verteilen (fanout)
- RabbitMQ speichert Messages in Queues, bis Consumer sie bestätigen (ACK)
- Monitoring über RabbitMQ Management UI mit Queue-Statistiken und Logs

RabbitMQ

- Producer: User-Service veröffentlicht Event „user.created“ an Exchange „UserEvents“
- Exchange (z. B. direct) routet Event in die Queues „EmailQueue“ und „AnalyticsQueue“
- Consumer 1: E-Mail-Service schickt Willkommensnachricht an Nutzer
- Consumer 2: Analytics-Service zählt Neuregistrierungen in Statistiken
- RabbitMQ sorgt für zuverlässige Verarbeitung beider Events, selbst bei Ausfällen
- Beispiel-Event: {"userId": 123, "name": "Anna", "timestamp": "2025-05-27T12:00:00Z"}

Danke!

Mathis Neunzig
dhbw@mathis-neunzig.de

